

Curso Inicial Front-End

Clase 24: REACT: FORMULARIOS CONTROLADOS

Módulo 3: Desarrollo con React.js y Ecosistema Moderno



Presentación

En **React**, los formularios funcionan distinto. En esta clase conoceremos los **Componentes Controlados**, una técnica donde **React** se convierte en la "única fuente de verdad" de lo que el usuario escribe. Aprenderemos a sincronizar el estado (**state**) con cada tecla presionada, permitiendo validaciones en tiempo real y una experiencia de usuario fluida y profesional.



Objetivos

- Comprender y aplicar la técnica de **Componentes Controlados** en formularios.
- Gestionar múltiples inputs de forma eficiente mediante un único objeto de estado.
- Implementar validaciones y capturar el envío de datos (**onSubmit**) de forma síncrona.



Bloques Temáticos

- Tema 1: Técnica de "Single Source of Truth" aplicada a formularios y validaciones.
 - El Input Controlado (Conceptos, Flujo y Handlers)
 - Componentes No Controlados
 - Patrón Profesional: Múltiples Inputs, Un Solo Estado
 - Ejercicio de Clase

En el HTML clásico, los formularios tenían mente propia: el navegador almacenaba los valores en el DOM de forma independiente. En React, cambiamos esta dinámica mediante la filosofía "Single Source of Truth" (Única Fuente de la Verdad). Esto significa que los valores de los inputs están centralizados exclusivamente en un solo lugar: nuestro estado (useState).

Esta técnica establece una relación síncrona "estado-input-evento" que garantiza la integridad de los datos:

1. El input visualiza siempre el valor actual del estado (value={estado}).
2. Ante cualquier interacción, el evento dispara una actualización del estado.
3. React re-renderiza el componente reflejando el nuevo valor.

Gracias a este flujo unidireccional, garantizamos que, en cada instante, el valor que ve el usuario en pantalla corresponda exactamente al valor almacenado en

nuestra memoria, hasta que finalmente se dispare el evento `onSubmit` para procesar los datos finales.

 **Recurso Oficial:** [React Docs - Respondiendo a inputs y formularios](#)

Tema 1: Técnica de "Single Source of Truth" aplicada a formularios y validaciones

1. El Input Controlado

Para que **React** tome el control absoluto de una caja de texto, necesitamos conectar dos cables:

1. **El Cable de Visualización (`value`):** Obliga a la pantalla a mostrar exclusivamente lo que dice nuestra memoria de estado.
2. **El Cable de Teclado (`onChange`):** Escucha cada milisegundo que el usuario aprieta una tecla, para actualizar nuestra memoria al instante.

Ejemplo de Código Básico:

```
import { useState } from 'react';

function Formulario() {
  const [email, setEmail] = useState('');

  return (
    <form>
      <input
        type="email"
        // Cable 1: Si el estado 'email' está vacío, la caja siempre
        // se verá vacía.
        value={email}
        // Cable 2: Cuando tipean una tecla (e.target.value),
        // actualizamos la memoria central.
        onChange={(e) => setEmail(e.target.value)}
      />
      <p>Estás escribiendo: {email}</p>
    </form>
  );
}
```

```
    </form>  
  );  
}
```

Fundamentos: Value, Flujo y Handlers

El valor real: ¿Qué es el `value`?

El atributo `value` no es solo una propiedad de HTML; es la **proyección visual del estado**. Cuando asignas `value={email}`, le estás diciendo al navegador: "No me importa lo que el usuario escribió, lo único que existe en pantalla es lo que vive dentro de mi estado `email`". Este valor es la ventana de lectura de tu Single Source of Truth: es la forma en que React le impone su 'verdad' al input, asegurando que lo que ves sea siempre lo que está en el estado.

El Flujo Unidireccional: ¿Por qué mi input está "congelado"?

Si omites el `onChange`, tu input se vuelve de **solo lectura**. Como React fuerza al input a mostrar siempre el valor del estado (que es `""`), al escribir, el evento ocurre pero el estado no cambia. En el siguiente render, React vuelve a obligar al input a mostrar `""`.

¿Evento en línea o Handler?

A medida que tu formulario crece, tienes dos formas de manejar el cambio:

- **Inline (Directo):** `onChange={(e) => setEmail(e.target.value)}`. Es útil para prototipos rápidos o inputs únicos.
- **Handler (Separado):** `const handleChange = (e) => setEmail(e.target.value);`. Esta es la práctica profesional.

¿Por qué usar un Handler?

1. **1. Separación de responsabilidades:** El JSX se mantiene limpio, enfocado en la estructura.
2. **2. Escalabilidad:** Si necesitas agregar lógica compleja (validaciones, transformaciones), modificas el handler sin tocar el HTML.

🔴 **Checkpoint de Comprensión (El Input de Piedra):** Estás programando a las 3:00 AM y, por cansancio, le pones a tu input el cable `value={email}`, pero te olvidas por completo de agregarle el evento `onChange`. Abres la pantalla, intentas escribir tu correo en la caja de texto, y escuchas el sonido del teclado, pero la pantalla no hace nada. La caja se siente "bloqueada" o rota. Teniendo en cuenta la filosofía de la "Única Fuente de Verdad", ¿por qué lógicamente **React** prohíbe que se dibuje lo que tipeaste? (Pista: Piensa quién le está diciendo a la pantalla qué mostrar, y si esa memoria alguna vez cambió).

2. Componentes No Controlados

A veces (por cuestiones de rendimiento), no quieres que **React** controle y redibuje la página completa con cada tecla que presionas. Imagina un simple "Buscador de Google". Solo te importa leer qué escribió el usuario en el momento exacto en que presionó el botón "Buscar", no necesitas leer letra por letra. Importante: Aquí decidimos abandonar la SSoT en el estado de React y dejar la verdad en el DOM. Es una decisión de diseño donde priorizamos rendimiento sobre el control centralizado de los datos.

Para esto usamos el "Componente No Controlado". Le devolvemos la libertad al **HTML** y usamos el hook `useRef` como un puntero láser que toca el DOM sin despertar a **React**.

Ejemplo de Código No Controlado (`useRef`):

```
import { useRef } from 'react';

function BuscadorRapido() {
  // 1. Creamos el puntero láser
  const inputPuntero = useRef(null);

  const ejecutarBusqueda = () => {
    // 3. Cuando hacemos clic, disparamos el láser hacia el HTML para
    leer el valor
    alert("Buscando en la base de datos: " +
inputPuntero.current.value);
  };

  return (
    <div>
      {/* 2. Le pegamos el láser al input físico */}
      <input type="text" ref={inputPuntero} placeholder="Busca
algo..." />
      <button onClick={ejecutarBusqueda}>Buscar</button>
    </div>
  );
}
```

Comparativa: ¿Cuál elijo?

Característica	Controlado (useState) ✓	No Controlado (useRef) ⚡
Fuente de Verdad	La Memoria de React (state).	La Memoria del Navegador (DOM).
Rendimiento	Redibuja la pantalla con CADA tecla que presionas.	Súper rápido, 0 redibujos hasta que envías el botón.
¿Cuándo usarlo?	El 95% del tiempo. Ideal para validaciones en vivo (ej: bloquear el	Solo para formularios gigantes muy lentos, o para

Característica	Controlado (useState) ✓	No Controlado (useRef) ⚡
	botón si la contraseña es débil), o autocompletado.	leer archivos (<code><input type="file"></code>).

3. Patrón Profesional: Múltiples Inputs, Un Solo Estado

Si tu formulario de registro tiene 10 campos (nombre, apellido, fecha, correo, etc.), crear 10 estados con 10 `useState` distintos se convierte en un código ilegible e inmanejable. Al agrupar todos los campos en un solo objeto, ese objeto se convierte en nuestra Single Source of Truth centralizada. Ahora, un solo lugar gobierna el estado de todos los inputs del formulario.

El estándar de la industria es agrupar todo en un solo Objeto, y usar el atributo `name` de **HTML** para actualizar todo con una sola función mágica.

Ejemplo Pro (Observa el Destructuring de `e.target`):

```
function FormularioRegistro() {
  const [form, setForm] = useState({
    nombre: "",
    pais: "Argentina",
    aceptaTerminos: false
  });

  const manejarCambios = (e) => {
    // Extraemos inteligentemente qué nombre tiene el input y qué
    valor teclearon
    const { name, value, type, checked } = e.target;

    // El 'Spread Operator' clona los datos viejos, y actualiza solo
    el campo que tocamos
    setForm({
      ...form,
      [name]: type === "checkbox" ? checked : value
    });
  };
}
```

```
};  
  
return (  
  <form>  
    { /* Obligatorio: El atributo 'name' debe llamarse IGUAL que en  
el estado */}  
    <input name="nombre" value={form.nombre}  
onChange={manejarCambios} />  
    <input type="checkbox" name="aceptaTerminos"  
checked={form.aceptaTerminos} onChange={manejarCambios} />  
  </form>  
);  
}
```

Documentación Oficial (Referencias)

- **HTML:** <https://developer.mozilla.org/es/docs/Web/HTML>
- **React:** <https://es.react.dev/>